Listing 6.7 Calling a retriever with a prompt message

```
import { StringOutputParser }
from "@langchain/core/output_parsers";
import { formatDocumentsAsString } from "langchain/util/document";
import {
  RunnablePassthrough,
  RunnableSequence,
} from "@langchain/core/runnables";

const retriever = vectorStore.asRetriever();
const chain = RunnableSequence.from([
  {
    context: retriever.pipe(formatDocumentsAsString),
    question: new RunnablePassthrough(),
  },
  prompt,
  module,
  new StringOutputParser(),
]);
const answer = await chain.invoke({
question: "What is artificial intelligence?" });
/*
' Artificial Intelligence (AI) is basically intelligence shown by machines...
*/
```

Imports the StringOutputParser for formatting output as strings

Converts the vector store into a retriever, allowing for document retrieval based on queries

Creates a sequence of runnable tasks to process input and generate an output

Sets up the context by retrieving documents and formatting them as strings

Let's consider what is happening here. First, we create a retriever from a vector store using `vectorStore.asRetriever()`. This retriever is responsible for fetching relevant documents based on the input query.

We use a `RunnableSequence`, which defines a pipeline of operations. Within this sequence, we use the retriever in combination with `formatDocumentsAsString` to convert all the `Document` instances to a list of strings. The `RunnablePassthrough` is used to pass the original question through the chain unmodified (think of it as an identity function).

The chain also includes a prompt, the language model itself, and a `StringOutputParser` to format the output. When invoked, this chain takes a question as input, retrieves relevant context, formats it along with the question into a prompt, sends it to the language model, and then parses the response.

There are a few things you must consider when working with chains that include a combination of retrievers:

- *The order of chain events is important.* Each component in the chain must receive the expected inputs from the previous step. For example, if a prompt expects both context and a question, these must be provided by the preceding chain elements.
- *Perform argument matching.* It's essential to ensure that the outputs of one chain component match the expected inputs of the next. Mismatches can lead to errors like the one mentioned here: `"Error: (f-string) Missing value for input context."`